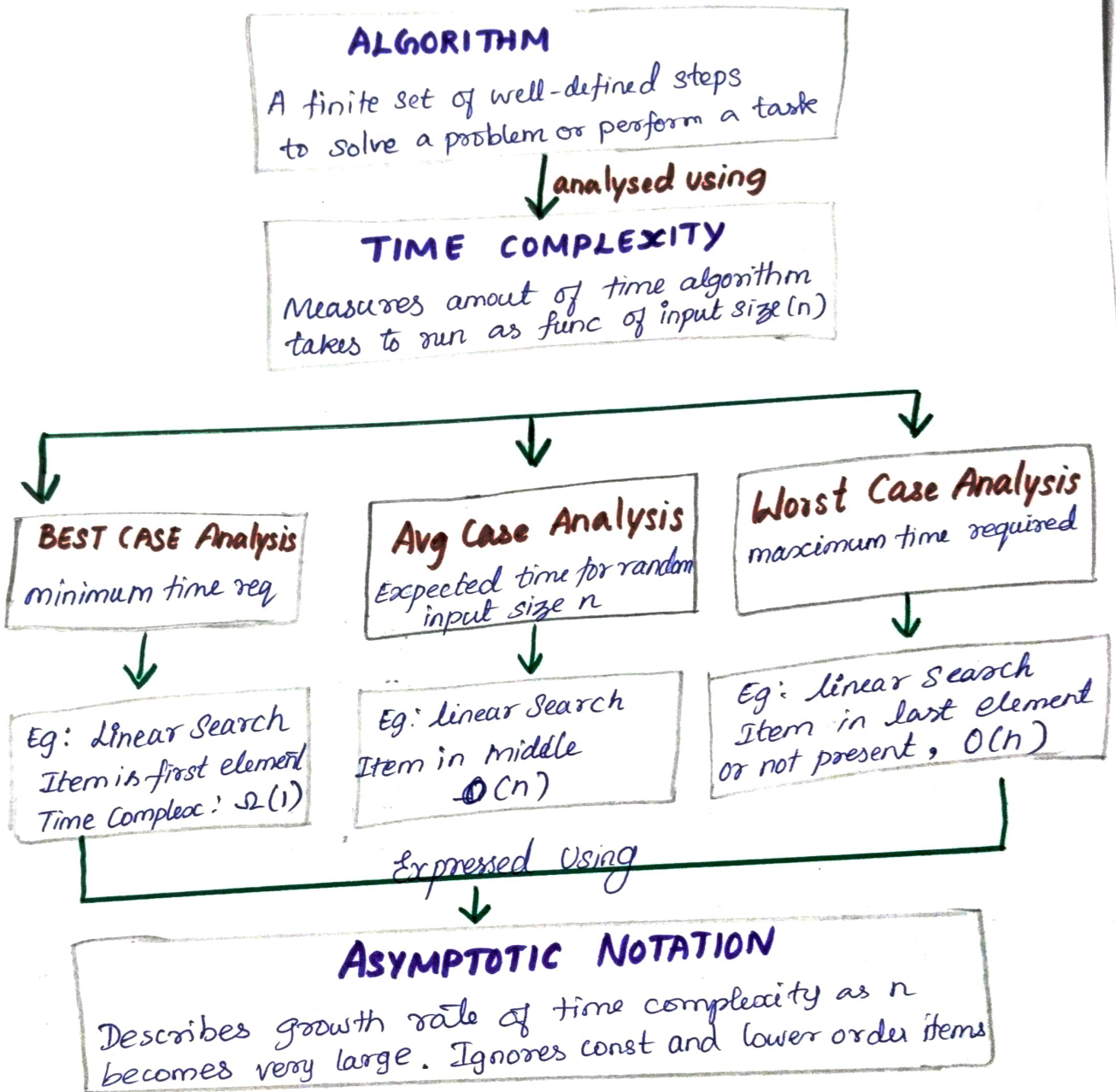


M. S. Divansha
192371010

CONCEPT MAPPING.

Q1)



Q2)

Divide & Conquer

Divide & Conquer

Break a problem of size n into a subprob. solve subproblems recursively and combine.

leads to

Recurrence Relation

The running time $T(n)$ satisfies a recurrence of form $T(n) = aT(n/b) + f(n)$

analyzed using

Master Theorem

Used to solve recurrence of form $T(n) = aT(n/b) + f(n)$ compare $f(n)$ with $n \log_b a$

Time Complexity

Applying Master Theorem and identifying case, we get asymptotic $T(n)$ in Θ notation $T(n) = \Theta(\dots)$

Where,
 a = no. of Subproblems
 n/b = size of each subprob.
 $f(n)$ = time for dividing & combining.

Case 1

$$f(n) = O(n^{d-\epsilon})$$

for some $\epsilon > 0$
 $f(n)$ grows strictly slower than n^d

Conditions

$$f(n) < O(n^{d-\epsilon})$$

for some $\epsilon > 0$ and sufficiently large n

Result

$$T(n) = \Theta(n^d)$$

The recurrence is dominated by recursive work

Case 2

$$f(n) = \Theta(n^d \log^k n)$$

for some $k \geq 0$
 $f(n)$ grows at same rate as n^d up to log factors.

Condition

$$f(n) = c \cdot n^d \log^k n$$

for constants $c > 0$
 $k \geq 0$ & sufficiently large n

Result

$$T(n) = \Theta(n^d \log^{k+1} n)$$

An extra logarithmic factor appears

Case 3

$$f(n) = \Omega(n^{d+\epsilon})$$

for some $\epsilon > 0$
 $f(n)$ grows strictly faster than n^d

$$f(n) \geq c \cdot n^{d+\epsilon}$$

$c > 0, \epsilon > 0$

$$T(n) = \Theta(f(n))$$

Identify pattern result form

Q3)

Recurrence relation $\rightarrow$ Substitution Method $\rightarrow$ Expansion
 $\rightarrow$ Complexity Derivation.

Recurrence Relation

Start with given recurrence relation for running time.
 $T(n) = aT(n/b) + f(n)$

Substitution Method

Assume a bound for $T(n)$ using a guess
 Use mathematical induction to test it
 Aim: $T(n) = O(g(n))$

Expansion

Repeatedly expand recurrence to observe pattern of work at each level
 $T(n) \rightarrow T(n/b) \rightarrow T(n/b^2) \rightarrow \dots$

Complexity Derivation

Combine result from Substitution/Expansion to get final closed form complexity.
 $T(n) = O(g(n))$

Represents how the problem of size n is broken into smaller subproblems

Reduces the recurrence to an inequality that can be proven

Substitution $\downarrow$ methods in detail

Guess

Make a initial guess for solution

Induction

Prove the guess is true for all $n \geq n_0$. Use mathematical induction.
 1. $n = n_0$
 2. $K < 0$, prove for n .

Verification

Verify that the guess satisfies recurrence inequality.
 $T(n) \leq aT(n/b)$

Conclusion

Conclude the bound & state the complexity
 $T(n) = O(g(n))$

Benefits

- Provides a systematic way to solve recurrence.
- Build intuition through guessing and proof.
- Ensures accuracy with verification.
- Leads to exact or tight complexity bounds.

Q(4)

Algorithm

A finite sequence of well-defined steps used to solve a problem.
Eg: searching, sorting.

Every alg works on some input. The nature of alg. determines what operations are performed.

Algorithm $\rightarrow$ Input Size $\rightarrow$ Growth Rate $\rightarrow$ Efficiency**Input Size (n)**

'n' represents the amount of input to algorithm.

Eg: No. of elements in an array.

As n increases, the algorithm has to process more data and perform more operation.

Growth rate

Describes how the no. of operations or running time increases as n increases.

Common growth rates:
 $O(1)$, $O(\log n)$, $O(n)$
 $O(n \log n)$, $O(n^2)$, $O(2^n)$

The pattern of increase in operations is called in growth rate of algorithm.

Efficiency

Measures how effectively an algorithm uses computational resources.

Includes:
 $\rightarrow$ Time Efficiency
 $\rightarrow$ Space Efficiency

Lower growth rate $\rightarrow$ Better scalability $\rightarrow$ Higher efficiency

Asymptotic notation

Used to describe growth of algorithm as n becomes very large.

Supported by

Scalability

Describes how well an alg. performs when input size increases.

Good Scalability

Algorithms with slow growth rates handle large inputs.

Eg: $O(\log n)$, $O(n)$, $O(n \log n)$

Poor Scalability

Alg. with fast growth rates

Eg: $O(n^2)$, $O(2^n)$, $O(n!)$

Big O

(Upper bound)

Big- Θ

(Tight bound)

Big- Ω

(Lower bound)

$$T(n) = O(g(n))$$

$$T(n) = \Theta(g(n))$$

$$O(n^2)$$

$$T(n) = \Theta(g(n))$$

$$\Omega(n \log n)$$

$$\Omega(n)$$